

```
else
```

This method decides what to do when the event happens.

3.6 Interfaces

When we said our program:

```
implements ActionListener
```

it was implementing a thing called an interface.

If you want to implement the interface `ActionListener`, it means that you must define a method called:

```
public void actionPerformed( ActionEvent e )  
in your class.
```

3.7 Recursion

For any programming problem, there are usually two alternative ways to solve it:

iteration - Iteration means repeating the same thing a certain number of times until the solution is achieved.

recursion - Recursion means having a method call itself. Or rather, it's better to think that it calls another copy of itself.

If we choose to call a method recursively, it means we are calling it numerous times until we arrive at the solution. Ironically, a recursive method can only actually solve the simplest "base" case. The other, more complex cases wait until the simplest base case is solved and then it works from the inside out until all the cases are solved.

Recursion Example: The Fibonacci Series

This is a Fibonacci series:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55...

It makes the next number in the sequence by adding together the previous two numbers.

```
import java.awt.*;  
import java.awt.event.*;  
import javax.swing.*;  
public class FibonacciTest extends JApplet implements  
ActionListener  
{  
    JLabel numLabel, resultLabel;  
    JTextField num, result;  
    public void init()
```